



Exp :9 Write the python to implement Travelling Salesman Problem

Aim:

To write a Python program to solve the Travelling Salesman Problem (TSP) and find the minimum cost to visit all cities and return to the starting city.

Algorithm:

Step 1: Represent the given cities and distances between them using an adjacency matrix (a 2D list).

Step 2: Choose a starting node (usually node 0) and create a list of all other nodes to visit.

Step 3: Generate all possible permutations of the remaining nodes to represent all possible paths.

Step 4: Initialize a variable to store the minimum path cost with a very large value (infinity)

Step 5: For each permutation (path):

- Calculate the total cost by summing the distances between consecutive nodes in the path.
- Add the distance from the last node back to the starting node.

Step 6: Compare the total cost of the current path with the minimum path cost. If it is smaller, update the minimum path cost.

Step 7: After checking all permutations, print the final minimum path cost.

Program:

```
from sys import maxsize

from itertools import permutations

def tsp(graph, s):

    V = len(graph)

    vertex = []

    for i in range(V):
```

```

        if i != s:
            vertex.append(i)

min_path = maxsize
next_permutation = permutations(vertex)

for i in next_permutation:
    current_pathweight = 0
    k = s

    for j in i:
        current_pathweight += graph[k][j]
        k = j

    current_pathweight += graph[k][s]
    min_path = min(min_path, current_pathweight)

return min_path

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

s = 0
print("Minimum cost to visit all cities:", tsp(graph, s))

```

OUTPUT:

```
Minimum cost to visit all cities: 80
```

Result: Thus, the program was executed and found that the output was found to be correct.